

Spec — WordPress Headless + React para el sitio público

Estado: spec de evaluación, sin construir. Ninguna línea de código de esta spec está implementada todavía.

Alcance acordado: solo el **sitio público** (listado de tours + flujo de reserva/checkout) pasa a un frontend React desacoplado. El panel de operación (**wp-admin**: Dashboard, Modo campo, Proveedores, Calendario, Configuración, etc.) **sigue siendo WordPress tal cual está hoy** — no se toca, no se migra, no entra en esta spec.

1. Por qué esto es viable sin reescribir el backend

Tres decisiones que ya están tomadas en el código actual hacen que este cambio sea mucho más barato de lo que sería en un plugin típico de WordPress:

- 1 **La REST API ya es la única puerta de entrada del frontend público.** El widget de React actual (`react-src/`) ya habla exclusivamente con `/wp-json/amir/v1/*` — nunca renderiza PHP del lado del servidor para su propia lógica. Un frontend headless separado consumiría exactamente la misma API, sin tocar el backend.
- 2 **El flujo de reserva no depende de sesión de WordPress.** Confirmado revisando el código: los endpoints públicos (`tours`, `bookings`, `availability`, `prices`, `wishlist`) tienen `permission_callback => '__return_true'` y **no verifican el nonce** (`wp_verify_nonce`) en ningún punto del flujo de reserva — el campo `nonce` que hoy se inyecta en `window.amirBooking` no lo consume ningún endpoint público. Esto significa que el checkout ya es **stateless** de facto: no hace falta resolver autenticación ni cookies cross-domain para que un frontend en otro dominio pueda reservar y pagar.
- 3 **No hay WooCommerce ni un sistema de carrito con estado en servidor** que atar — cada reserva es una fila en `wp_amir_bookings` identificada por `booking_ref` + `access_token`, no una sesión de PHP.

Dicho esto, hay tres cosas que **sí** están acopladas hoy a que WordPress renderice la página, y que hay que resolver — el punto 3 (gaps) las detalla.

2. Qué ya sirve tal cual, sin tocar nada

Toda la superficie de datos que necesita un catálogo de tours + checkout ya existe como REST pública:

GET	/wp-json/amir/v1/tours	Listado
GET	/wp-json/amir/v1/tours/{id}?lang=es	Detalle (contenido ya viene i18n-resuelto)
GET	/wp-json/amir/v1/tours/{id}/schedules	
GET	/wp-json/amir/v1/tours/{id}/prices	
GET	/wp-json/amir/v1/availability/month day schedules	
POST	/wp-json/amir/v1/bookings/quote	Cotización con cupón
POST	/wp-json/amir/v1/bookings	Crear reserva + iniciar pago (Stripe/MP)
POST	/wp-json/amir/v1/bookings/{id}/confirm-payment	Confirmación verificada server-side
GET	/wp-json/amir/v1/bookings/{ref}?token=...	Ver una reserva (para la pantalla de confirmación)
POST	/wp-json/amir/v1/bookings/stripe-webhook	Server-to-server, no lo toca el frontend
POST	/wp-json/amir/v1/bookings/mercadopago-webhook	Ídem
GET	/wp-json/amir/v1/tours/upcoming + POST .../wishlist	Lista de interés

El modelo de pago tampoco cambia: los webhooks de Stripe/Mercado Pago siguen pegándole directo a WordPress (comunicación servidor-a-servidor, no pasan por el navegador), así que ningún secreto de pasarela se mueve ni se expone nuevo.

3. Gaps reales — qué falta para que esto funcione

3.1 Config de arranque del widget — ■ resuelto (v2.8.0)

Hasta v2.7.0, todo lo que el widget de React necesita para arrancar (`stripePk`, moneda activa, idiomas activos, teléfono de WhatsApp, nombre de marca, config de Meta Pixel/GA4, texto de política de cancelación) se inyectaba solo server-side vía `wp_localize_script()` en `class-shortcodes.php` — un objeto `window.amirBooking` que solo existía porque WordPress renderizó la página.

Ya existe `GET /wp-json/amir/v1/config (includes/api/class-config-controller.php)`, público, con ese mismo payload — ver el shape completo en [GUIA-INTEGRACION-API.md § 2](#).

3.2 Tema visual del widget — ■ resuelto (v2.8.0)

`Core\WidgetTheme::render_inline_css()` sigue generando el CSS inyectado server-side para el sitio actual — sin cambios ahí. Lo nuevo es `WidgetTheme::resolved_values()`, que expone los mismos valores resueltos (color + variantes, fuente, radio) como datos en vez de texto CSS, incluido en el mismo `GET /config` de arriba bajo la clave `theme`. Un frontend headless los aplica como sus propias CSS variables, tal como ya hace `react-src/src/styles/widget.css` con `--ab-*`.

3.3 SEO — JSON-LD y metadata (esfuerzo medio)

`Core\StructuredData::tour_schema()` arma el `TouristTrip` JSON-LD hoy **dentro de** `templates/single-amir_tour.php`, en PHP, en el momento en que WordPress renderiza la página. En headless, quien arma el HTML final es el frontend (Next.js u otro), así que esta lógica necesita moverse a un lugar consumible: la opción más simple es un endpoint `GET /wp-json/amir/v1/tours/{id}/schema` que devuelva el JSON-LD ya armado (reusa `StructuredData::tour_schema()` server-side, cero lógica duplicada), y el frontend lo inyecta en un `<script type="application/ld+json">`. Título, meta description, Open Graph, sitemap — todo eso pasa a responsabilidad del frontend headless (Next.js lo resuelve bien con generación estática, pero es trabajo nuevo, hoy WordPress lo da gratis vía el tema/SEO plugin si hay uno instalado).

3.4 CORS (bloqueante, esfuerzo bajo pero fácil de subestimar)

Hoy no hay ninguna configuración de CORS en el plugin — la API se sirve desde el mismo origen que la página que la consume. Un frontend headless en otro dominio (o incluso otro subdominio) va a disparar preflight `OPTIONS` en cualquier `POST` (crear reserva, confirmar pago, cotizar). Hace falta agregar manejo explícito de CORS (`Access-Control-Allow-Origin` limitado al dominio del frontend headless — nunca `*` en un endpoint que crea reservas — más `Allow-Methods/Allow-Headers` para el preflight).

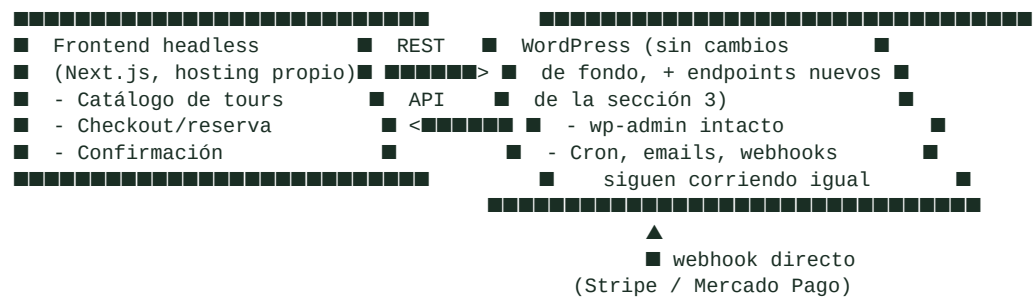
3.5 Elementor y páginas armadas visualmente (a confirmar con el negocio, no técnico)

El plugin tiene integración con Elementor (`includes/elementor/`: widgets "Botón Reservar Tour", "Tarjeta de Tour", Dynamic Tags, compat con Loop Builder) para que alguien sin tocar código arme grillas o páginas de marketing. **Esto es exclusivo de WordPress renderizando la página** — no tiene ningún equivalente en un frontend headless. Antes de avanzar, vale la pena confirmar: ¿hoy se usa Elementor para algo más que el template default de tour? Si el cliente (o vos) arma landing pages o ediciones visuales ahí, pasar a headless significa que **cualquier cambio visual futuro en el sitio público requiere un desarrollador tocando React** — se pierde la edición visual sin código. Es la única pérdida de esta migración que no es "trabajo pendiente", es una capacidad que no se reemplaza.

3.6 Multi-idioma (sin gap real)

`content_i18n` ya resuelve el contenido por idioma server-side y lo devuelve vía `?lang=`. El frontend headless solo necesita decidir su propio ruteo por idioma (ej. `/en/tours/...` en Next.js) y pasar el parámetro — no hay lógica de traducción que reconstruir.

4. Arquitectura propuesta



- **Next.js** (o similar) como frontend, hosteado aparte de WordPress (Vercel, o el mismo servidor si hace falta simplificar operación).
- **WordPress sigue siendo el sistema de verdad:** base de datos, panel de operación, cron, emails, integración de pagos. No se "apaga" nada de lo que existe — se le agrega una API más completa.
- El widget de checkout en React (`BookingWidget.jsx`) es candidato a **reusarse casi tal cual** dentro de Next.js — hoy ya es un componente autónomo que solo habla por HTTP con la API; portarlo no es reescribirlo desde cero, es cambiar cómo se monta (de `createRoot` sobre un `data-attribute` a un componente normal de la app Next.js) y resolver el `config` bootstrap por fetch en vez de `window.amirBooking`.

5. Plan de migración por fases

Fase	Qué incluye	Esfuerzo estimado
0 — Preparar el backend	~~Endpoint <code>/config</code> ~~ (■ ya construido, v2.8.0). Falta: endpoint <code>/tours/{id}/schema</code> , CORS explícito para el dominio del frontend headless. Cero riesgo para lo existente — son endpoints nuevos, no tocan los que ya usa el widget actual.	1-2 días
1 — Catálogo público	Next.js: listado de tours (<code>/tours</code>) + página de detalle, consumiendo <code>/tours</code> , <code>/tours/{id}</code> , <code>/tours/{id}/schema</code> . SSG/ISR para SEO. Sin checkout todavía — el botón de reservar puede seguir apuntando al widget actual en WordPress mientras tanto (convivencia).	1-2 semanas
2 — Checkout headless	Portar <code>BookingWidget.jsx</code> / <code>PayBooking.jsx</code> a componentes Next.js reales, resolviendo <code>/config</code> en vez de <code>window.amirBooking</code> . Probar Stripe y Mercado Pago de punta a punta en el dominio nuevo.	1-2 semanas

Fase	Qué incluye	Esfuerzo estimado
3 — Corte	DNS/dominio del sitio público apunta al frontend headless. WordPress pasa a responder solo <code>/wp-json/*</code> + <code>/wp-admin/*</code> para el público (o directamente deja de resolver como sitio visible).	1-2 días + monitoreo

Estimación total: **4-6 semanas** de una persona con el perfil de tu amiga (React sólido), asumiendo que el trabajo de la Fase 0 (backend) lo hacés vos o coordinado en paralelo.

6. Trade-offs — para decidir, no solo para ejecutar

A favor:

- DX 100% React para todo el desarrollo futuro del sitio público — nada de PHP para tu amiga.
- Next.js da SSG/ISR real, que WordPress con este stack (sin plugin de cache dedicado) no da hoy.
- Deploys del frontend desacoplados del deploy del plugin (menos riesgo al iterar en diseño).

En contra:

- Dos superficies de deploy en vez de una (el frontend Next.js + el plugin WordPress) — más piezas móviles operativamente.
- Se pierde Elementor como editor visual del sitio público (sección 3.5) — cualquier cambio de layout pasa a requerir un push de código.
- Trabajo real de "pegamento" nuevo (CORS, endpoint de schema) que hoy no existe — no es gratis, aunque es acotado (el endpoint de config, la otra pieza de esta categoría, ya está resuelto).
- WordPress se queda igual de "acoplado" en el panel de admin (`$wpdb` directo, `get_option`) — este spec no avanza nada hacia independizar el backend en sí, solo desacopla el frontend público. Si el objetivo de fondo es eventualmente dejar de depender de WordPress del todo, esto es un primer paso razonable pero parcial.

7. Recomendación

Es viable con esfuerzo acotado (4-6 semanas) porque el 90% del trabajo pesado — la REST API pública, el modelo de reserva sin sesión, el widget de React ya desacoplado de PHP — ya está hecho. El costo real no es técnico, es el punto 3.5: confirmar cuánto se apoya hoy el negocio en poder editar el sitio público sin un desarrollador. Si la respuesta es "poco o nada", esto tiene sentido. Si Elementor se usa activamente para armar páginas de marketing, vale la pena decidir eso explícitamente antes de arrancar, no descubrirlo a mitad de la Fase 1.